

Lecture 14

Error Handling, Part 2 Exception Safety and Best Practices

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Exception Safety

A function can't *complete normally* when it:

- Can't compute its **desired return data**
- Can't ensure it's **postconditions** (i.e. effects)

1. No-Fail Guarantee

The function always completes normally.

2. Strong Guarantee

If the function can't complete normally, no state is modified.

3. Basic Guarantee

If the function can't complete normally, no invariants are broken.

4. You're on your own, kid.



Exception Safety

- What happens if an exception is thrown?
- This version provides **the basic guarantee**.
 - The function didn't complete normally (we didn't get our copy of the back element), but it has been removed.

```
template <typename T>
class vector {
private:
    T *elts;           // points to dynamic array
    size_t capacity;   // capacity of elts array
    size_t elts_size;  // number of slots used in elts

public:

    T pop_back() {
        assert(elts_size > 0);
        return elts[--elts_size]; // may throw (copy ctor for T)
    }
}
```

C++



Exception Safety

- What happens if an exception is thrown?
- This version provides **the no-fail guarantee**.
 - The exception thrown by the “pop and return” version was a **vexing exception**.

```
template <typename T>
class vector {
private:
    T *elts;           // points to dynamic array
    size_t capacity;   // capacity of elts array
    size_t elts_size;  // number of slots used in elts

public:

    void pop_back() {
        assert(elts_size > 0);
        --elts_size;
    }

    T & back() {
        assert(elts_size > 0);
        return elts[elts_size - 1]; // return by reference
    }
}
```

C++



Exception Safety

- What happens if an exception is thrown?
- This version provides **no exception safety**.
 - Object may be in an inconsistent/invalid state with only some members updated.

```
template <typename T>
class vector {
private:
    T *elts;           // points to dynamic array
    size_t capacity;   // capacity of elts array
    size_t elts_size;  // number of slots used in elts

public:
    vector & operator=(const vector &rhs) {
        if (this == &rhs) { return *this; }

        capacity = rhs.capacity;
        elts_size = rhs.elts_size;

        delete[] elts;
        elts = new T[rhs.capacity]; // may throw (bad_alloc)
        for (int i = 0; i < rhs.elts_size; ++i) {
            elts[i] = rhs.elts[i]; // may throw (assn op for T)
        }

        return *this;
    }
}
```

C++



Exception Safety

- What happens if an exception is thrown?
- This version is almost able to provide the **strong guarantee**. But it leaks memory.
 - We'll come back to this next lecture...

```
template <typename T>
class vector {
private:
    T *elts;           // points to dynamic array
    size_t capacity;   // capacity of elts array
    size_t elts_size;  // number of slots used in elts

public:
    vector & operator=(const vector &rhs) {
        T *temp = new T[rhs.capacity]; // may throw (bad_alloc)

        for (int i = 0; i < rhs.elts_size; ++i) {
            temp[i] = rhs.elts[i]; // may throw (assn op for T)
        }

        delete[] elts;
        elts = temp;
        capacity = rhs.capacity;
        elts_size = rhs.elts_size;

        return *this;
    }
}
```

C++



Case Study 1

- What tasks in the overall system might this function support?
- Consider 3 versions...
 - Expressive Power
 - Complexity
 - Obscurity
 - Depth

```
/**
 * Delete a player from the database by their ID.
 * @param player_id The ID of the player to delete.
 * → @throws if player_id does not exist in the database.
 */
export async function deletePlayer(player_id: string) {
```

```
/**
 * Delete a player from the database by their ID.
 * @param player_id The ID of the player to delete.
 * → @returns A Result<void, DBError>
 */
export async function deletePlayer(player_id: string) {
```

```
/**
 * Delete a player from the database by their ID.
 * @param player_id The ID of the player to delete.
 * → @returns True if player was deleted, else false.
 */
export async function deletePlayer(player_id: string) {
```

“Define errors out of existence” - John Osterhaut

Encapsulate Error Handling



Coding Style: Exceptions

- Control flow along the “**Happy Path**” is clearly encoded.
- The **error paths** are **implicit** - anything thrown aborts the happy path and jumps to a catch or propagates outward.

```
while(true) {  
  try {  
    await this.autosave?.(game.config, game.getState()); // DBError  
  
    const action = await view.promptAction(); // ParseError, QuitEx  
    this.action_history.push(action);  
    this.game.performAction(action); // InvalidActionError  
    this.view.render();  
  
    if (this.game.getStatus().status === "finished") { return; }  
  }  
  catch (error: unknown) {  
    if (error instanceof ParseError) {  
      console.log(colors.red(`Invalid input: ${error.message}`));  
    } else if (error instanceof QuitEx) {  
      console.log("Thanks for playing!");  
      return;  
    } else if (error instanceof InvalidActionError) {  
      console.log(colors.red(`Invalid action: ${error.message}`));  
    } else {  
      throw error; // re-throw unexpected errors  
    }  
  }  
}
```


Coding Style: Errors as Values

- All control flow is **explicit**, including the error paths.
- The **happy path** is less clear. Opinions vary on whether this is a feature or bug.
- No errors propagate unless you specify it.

```
while(true) {  
  const prompt_res = await this.view.promptForAction();  
  switch (prompt_res.kind) {  
  
    case "valid_action":  
      this.action_history.push(prompt_result.action);  
      const act_res = this.game.performAction(prompt_result.action);  
      if (!act_res.ok) {  
        console.log(colors.red(`Invalid Action: ${act_res.error}`));  
      } else {this.view.render(); }  
  
      break;  
  
    case "parse_failed":  
      console.log(colors.red(`Invalid input: ${prompt_res.message}`));  
      await tui.prompt("Press Enter to continue...");  
      break;  
  
    case "quit":  
      console.log("Thanks for playing!");  
      return;  
  
    default: assertNever(prompt_res);  
  }  
  if (this.game.getStatus().status === "finished") { return; }  
}
```

Version 1 - Involves more nesting than is ideal.

Coding Style: Errors as Values

- All control flow is **explicit**, including the error paths.
- The **happy path** is less clear. Opinions vary on whether this is a feature or bug.
- No errors propagate unless you specify it.

```
while(true) {
  const prompt_res = await this.view.promptForAction();
  if (prompt_res.kind === "quit") { // bail out
    console.log("Thanks for playing!");
    return;
  }

  if (prompt_res.kind === "parse_failed") { // oof
    console.log(colors.red(`Invalid input: ${prompt_res.message}`));
    await this.view.prompt("Press Enter to continue...");
    continue;
  }

  // happy path
  prompt_res.kind satisfies "valid_action"; // exhaustiveness check
  this.action_history.push(prompt_result.action);
  const act_res = this.game.performAction(prompt_result.action);

  if (!act_res.ok) { // ouch
    console.log(colors.red(`Invalid Action: ${act_res.error}`));
    continue;
  }

  // happy path again
  this.view.render();
  if (this.game.getStatus().status === "finished") { return; }
}
```

Version 2 - Uses guards to clean up control flow.



Case Study 2

- Where should we catch and handle?
- Critically review this code. What can be improved?
- How would this look if we used “errors as values”?

```
const tui = new PuzzleController(game, view, autosave_minesweeper);
```

```
while(true) { // game loop in the TUI controller
  await this.autosave?.(this.game.config, this.game.getState());
  const action = await this.view.promptForAction();
```

```
async function autosave_minesweeper(cfg: MSConfig, st: MSState) {
  const player = Player.currentPlayer();
  if (player) { await player.setAutosave({ kind: "ms", cfg, st }); }
```

```
export class Player {
  public setAutosave(save: PuzzleSave): Promise<void> {
    this._autosave = save; // cache in memory
    return Autosave.saveByPlayerId(this.player_id, save);
```

```
namespace Autosave { // Helper within players.ts
  export async function saveByPlayerId(id: string, save: PuzzleSave){
    await updatePlayerAutosave(id, JSON.stringify(save));
```

```
/** @throws if player_id does not exist in the database. */
export async function updatePlayerAutosave(id: string, save: string){
  return updatePlayerById({id, save});
}
```

```
/** @throws if player does not exist in the database. */
export async function updatePlayerById( ... ) { ... }
```

Errors + Inversion of Control

- What do you think about this code?
- Principle:
 - Observer callbacks should never intentionally throw exceptions.
 - Exception: can still throw exceptions as a defensive programming/debugging technique.

```
interface SudokuObserver {
  onChange(change: StateChange): void;
}

export class SudokuGame {
  private observers: SudokuObserver[] = [];

  public addObserver(observer: SudokuObserver) {
    this.observers.push(observer);
  }

  public attemptAction(action: SudokuAction): ActionResult {

    // Process the action and update game state
    // ...

    this.observers.forEach(observer => {
      try {
        observer.onChange();
      } catch (error) {
        console.error("Observer threw an error:", error);
        // What should we do here
      }
    });
  }
}
```

Break Time

bit.ly/4sKpEKr

